머신러닝 · 앙상블

앙상블
랜덤 포레스트와 부스팅

결정 트리 여러 개를 묶어 더 강한 모델 만들기 — 배깅 · 부스팅 · LightGBM

v12 · 2026.10

© 2026. LDJ all rights reserved.

목 차

01 **앙상블 기법**
배깅과 부스팅

02 **랜덤 포레스트**
원리 · 만드는 5단계 · 매개변수

03 **그래디언트 부스팅**
오차를 차례로 보완 · 그림 · 매개변수

04 **부스팅 라이브러리 · LightGBM**
XGBoost · LightGBM · CatBoost · HistGB

05 **두 모델의 비교**
언제 무엇을 쓰나 · 특성 중요도

06 **여러 가지 모델**
지금까지 배운 모델 한눈에

© 2026. LDJ all rights reserved.

01 앙상블 기법

여러 모델을 묶어 더 강한 모델을 만드는 두 가지 방법

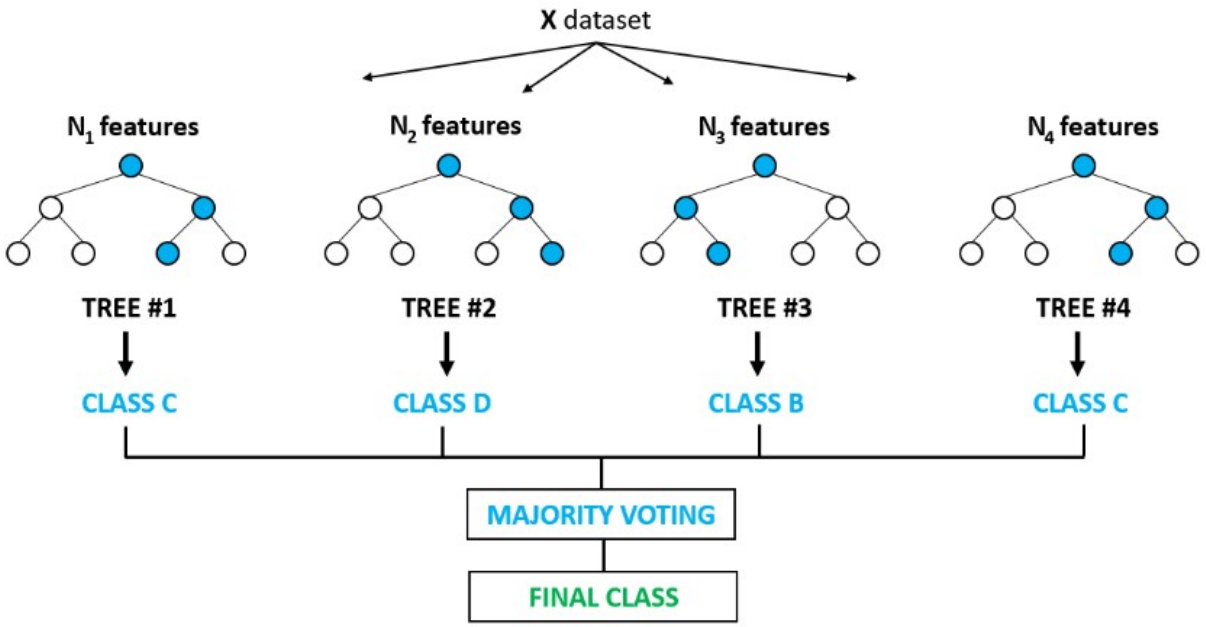


배깅의 대표가 랜덤 포레스트, **부스팅**의 대표가 그래디언트 부스팅이다. 두 방법 모두 기본 부품으로 **결정 트리**를 쓴다.

© 2026. LDJ all rights reserved.

02 랜덤 포레스트

조금씩 다른 결정 트리 여러 개의 묶음



결정 트리의 약점

- 학습 데이터에 **과대적합**되기 쉽다

랜덤 포레스트의 해법

- 데이터·특성을 조금씩 다르게 써서 트리를 **여러 개** 만든다
- 트리 하나는 데이터 일부에 과대적합되지만, 결과를 합치면 줄어든다

예측 합치기

- 분류: 그림처럼 **다수결**(원 논문)
- 회귀: 트리 예측값의 **평균**

© 2026. Lim all rights reserved.

02 랜덤 포레스트의 원리

왜 여러 트리를 묶나

핵심 아이디어

- 트리 하나는 데이터의 일부에 **과대적합**되기 쉽다.
- 조금씩 다른 트리를 많이 만들어 결과를 **평균**하면 과대적합이 줄어든다.

좋은 숲의 두 조건

- 트리 하나하나가 타깃을 **잘 예측**해야 한다.
- 트리끼리 **조금씩 달라야** 합쳤을 때 효과가 난다.

트리를 서로 다르게 만드는 두 가지 무작위성

- ① **데이터 포인트**를 무작위로 뽑는다 (부트스트랩 샘플)
- ② 노드마다 **특성 일부**를 무작위로 골라 그중 최선의 분할을 찾는다

© 2026. Lim all rights reserved.

02 랜덤 포레스트 만드는 5단계

1. **트리 개수를 정한다**
   n_estimators — scikit-learn 기본값 **100**

2. **부트스트랩 샘플을 뽑는다**
   n개 데이터에서 **중복을 허용해** n번 뽑기 → 빠지는 포인트, 여러 번 들어가는 포인트가 생긴다

3. **노드마다 특성 일부를 무작위로 고른다**
   max_features — 값이 **크면** 트리들이 비슷해지고, **작으면** 많이 달라진다

4. **샘플마다 결정 트리를 학습한다**
   트리마다 데이터와 특성이 달라 서로 다른 트리가 된다

5. **결과를 합친다**
   회귀는 예측값 **평균**, 분류는 클래스 확률 **평균**(scikit-learn)

© 2026. Lim all rights reserved.

02 랜덤 포레스트로 예측하기

회귀와 분류

회귀

- 각 트리가 예측한 값의 **평균**이 최종 예측

분류

- 원 논문: 트리마다 한 표씩 **다수결**
- scikit-learn: 트리별 **클래스 확률을 평균**해 가장 높은 클래스

두 방식이 다를 수 있다 (예시 가정)

- 트리 3개가 클래스 A일 확률을 0.9 · 0.4 · 0.4로 냈다면
- 확률 평균은 0.57 → **A** / 다수결은 B가 2표 → **B**

© 2026. Lim all rights reserved.

02 랜덤 포레스트의 장단점

장점 · 단점

장점

- 트리 하나보다 **덜 과대적합**되고 결정 경계가 좋다
- 매개변수를 많이 조정하지 않아도 잘 작동한다
- 특성 **스케일 조정이 필요 없다**

단점

- 차원이 매우 높고 **희소한** 데이터(예: 텍스트)에는 잘 맞지 않는다
- 선형 모델보다 메모리를 많이 쓰고 학습·예측이 느리다

© 2026. Lim all rights reserved.

02 랜덤 포레스트 - 매개변수

scikit-learn 1.9 기본값 (확인일 2026-10-13)

	뜻	기본값
n_estimators	만들 트리 개수	**100** (0.22부터, 이전 10)
max_features	노드마다 살펴볼 특성 수	분류 **"sqrt"** · 회귀 **1.0**(=전체 특성)
n_jobs	사용할 CPU 코어 수	None(=1) · **-1이면 모든 코어**
random_state	무작위 뽑기를 고정해 결과 재현	None

© 2026. Lim all rights reserved.

03 그래디언트 부스팅

핵심 특징

1. **여러 결정 트리를 차례로 묶는다**
   분류와 회귀 모두에 쓸 수 있다

2. **앞 트리의 오차를 보완한다**
   랜덤 포레스트처럼 따로 만들지 않고, 남은 오차를 다음 트리가 줄인다

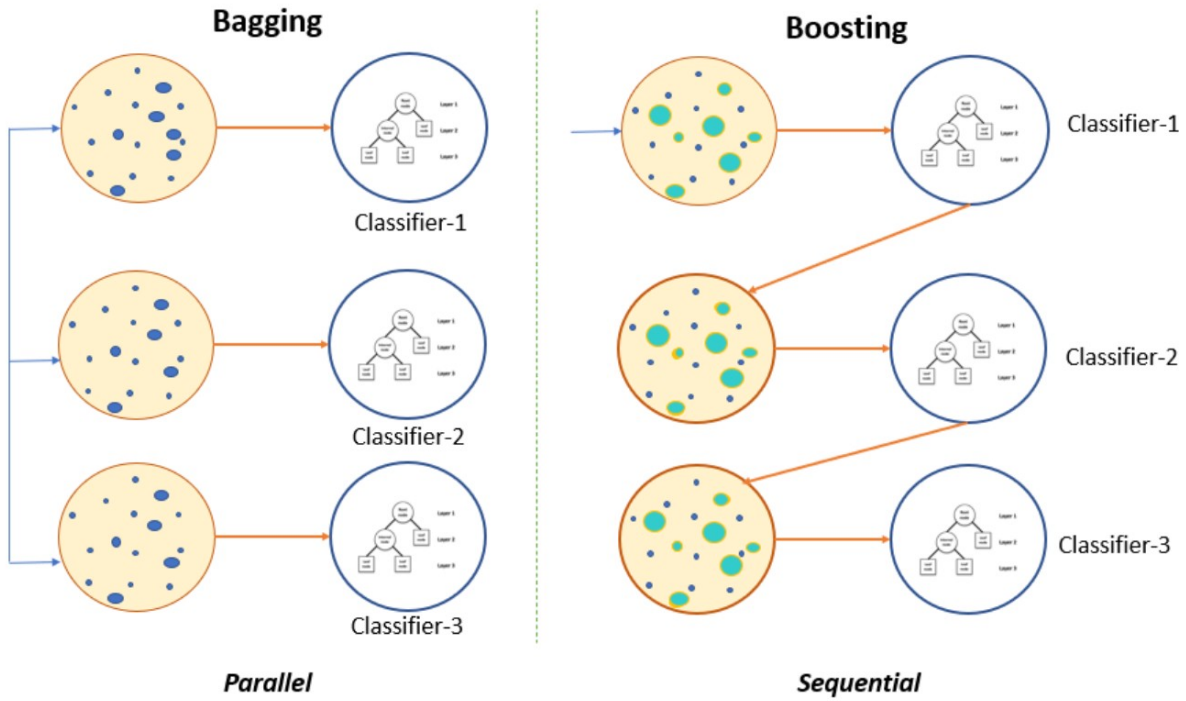
3. **얕은 트리 + 강한 사전 가지치기**
   기본 설정(subsample=1.0)에서는 무작위 샘플링이 없다

4. **매개변수에 민감하다**
   잘 조정하면 정확도가 높지만, 트리를 너무 많이 쓰면 **과대적합**될 수 있다

© 2026. Lim all rights reserved.

03 배깅과 부스팅 그림으로 보기

따로 학습(병렬) vs 차례로 학습(순차)



Source: https://pluralsight2.imgix.net/guides/81232a78-2e99-4ccc-ba8e-8cd873625fdf_2.jpg

배깅 (Parallel)

- 샘플을 따로 뽑아 분류기를 **동시에** 학습
- 예: 랜덤 포레스트

부스팅 (Sequential)

- 앞 분류기의 결과(틀린 부분)를 반영해 다음 분류기를 **차례로** 학습
- 예: 그래디언트 부스팅

참고: https://medium.com/@skilltohire/the-beginners-guide-for-gradient-boosting-e5c67584240e

© 2026. LDJ all rights reserved.

03 그래디언트 부스팅의 학습 방식

오차를 줄여 가는 과정

1. **손실 함수를 정한다**
   예측이 얼마나 틀렸는지 재는 기준

2. **다음 트리는 남은 오차를 학습한다**
   손실을 줄이는 방향(음의 기울기)을 새 트리가 맞춘다 — 경사 하강법과 같은 원리

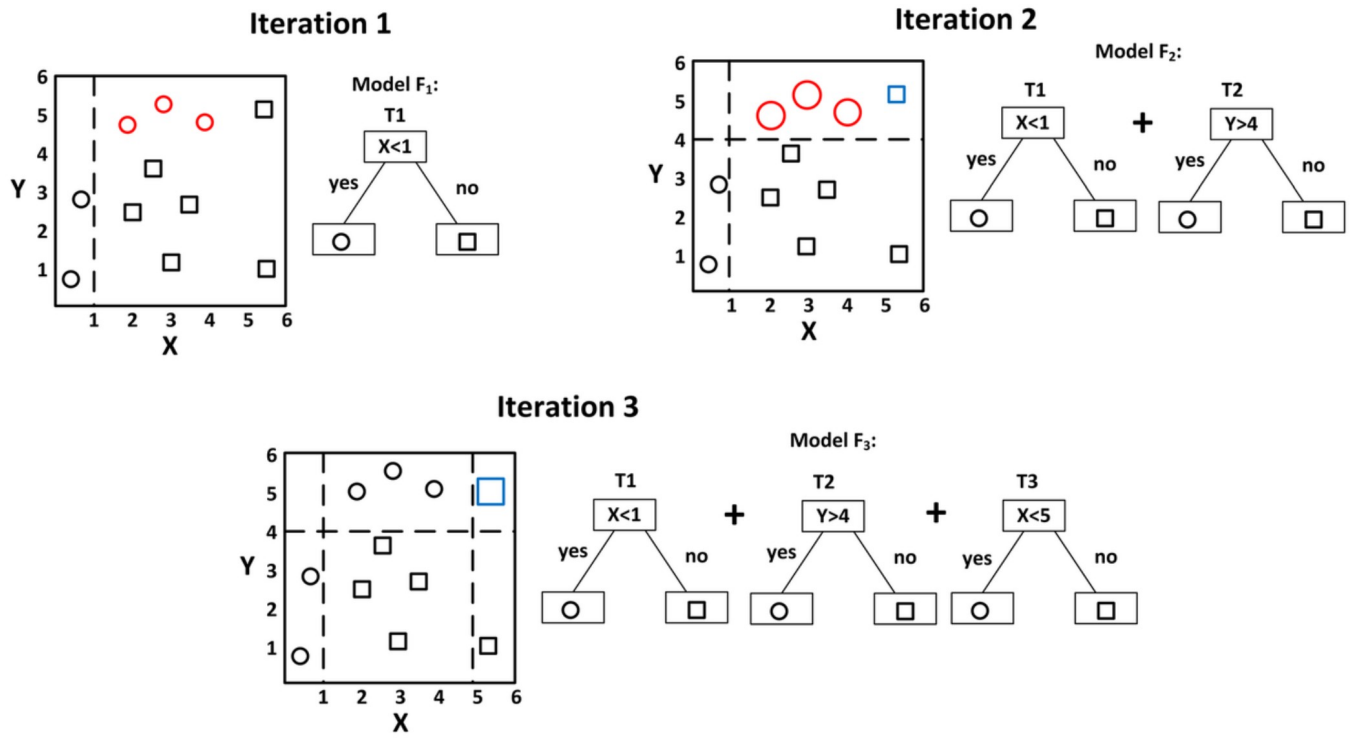
3. **learning_rate로 보정 세기를 정한다**
   크면 오차를 강하게 보정해 모델이 복잡해진다

4. **얕은 트리라 예측이 빠르고 메모리를 적게 쓴다**
   대회와 업계에서 널리 쓰인다

© 2026. Lim all rights reserved.

03 그래디언트 부스팅 그림으로 보기

트리를 하나씩 더할수록 경계가 정교해진다



Iteration 1 → 2 → 3

트리 T1(X<1) 하나로 시작해, T1이 놓친 부분을 T2(Y>4)가, 남은 오차를 T3(X<5)가 차례로 보완한다

참고: https://www.researchgate.net/figure/A-simple-example-of-visualizing-gradient-boosting_fig5_326379229

© 2026. LDJ all rights reserved.

03 그래디언트 부스팅의 장단점

장점 · 단점

장점

- 특성 **스케일 조정이 필요 없다**
- 이진 특성과 연속 특성이 섞여 있어도 잘 동작한다

단점

- 매개변수를 **잘 조정해야** 한다
- 트리를 차례로 만들어 학습 시간이 길다
- 희소한 고차원 데이터에는 잘 맞지 않는다

© 2026. Lim all rights reserved.

03 그래디언트 부스팅 - 매개변수

GradientBoostingClassifier 기본값 (scikit-learn 1.9, 확인일 2026-10-13)

	뜻	기본값
learning_rate	앞 트리 오차를 얼마나 강하게 보정할지	**0.1**
n_estimators	트리 개수 — 시간·메모리에 맞춰 정한 뒤 learning_rate를 조정, 너무 많으면 **과대적합** 가능	**100**
max_depth	트리 하나의 최대 깊이 (보통 5 이하로 얕게)	**3**
subsample	트리마다 쓸 데이터 비율 (1.0 미만이면 무작위성 추가)	1.0
n_iter_no_change	검증 점수가 안 오르면 일찍 멈춤	None (끔)

© 2026. Lim all rights reserved.

04 부스팅 라이브러리

대표 구현 한눈에 (확인일 2026-10-13)

	특징	설치
XGBoost	2.0부터 **hist**(히스토그램) 방식이 기본, 트리는 기본적으로 **깊이 우선 (depthwise)** 성장	pip install xgboost
LightGBM	히스토그램 방식이라 **빠르고 메모리를 덜 쓴다**, **leaf-wise** 성장	pip install lightgbm
CatBoost	기본 트리가 **대칭 트리**(SymmetricTree), 범주형 특성 처리를 내장	pip install catboost
HistGradientBoosting	scikit-learn 내장, **결측치(NaN)** 그대로 학습, 샘플 1만 개 이상에서 빠름	추가 설치 없음

© 2026. LDJ all rights reserved.

04 HistGradientBoosting

scikit-learn에 들어 있는 히스토그램 기반 부스팅 (확인일 2026-10-13)

1

큰 데이터에서 빠르다

샘플이 **1만 개 이상**이면 GradientBoosting보다 훨씬 빠르다

2

결측치(NaN)를 그대로 받는다

분할마다 NaN 샘플을 왼쪽·오른쪽 중 어디로 보낼지 학습한다

3

조기 종료가 기본

early_stopping='auto' — 샘플이 **1만 개를 넘으면** 자동으로 켜진다

4

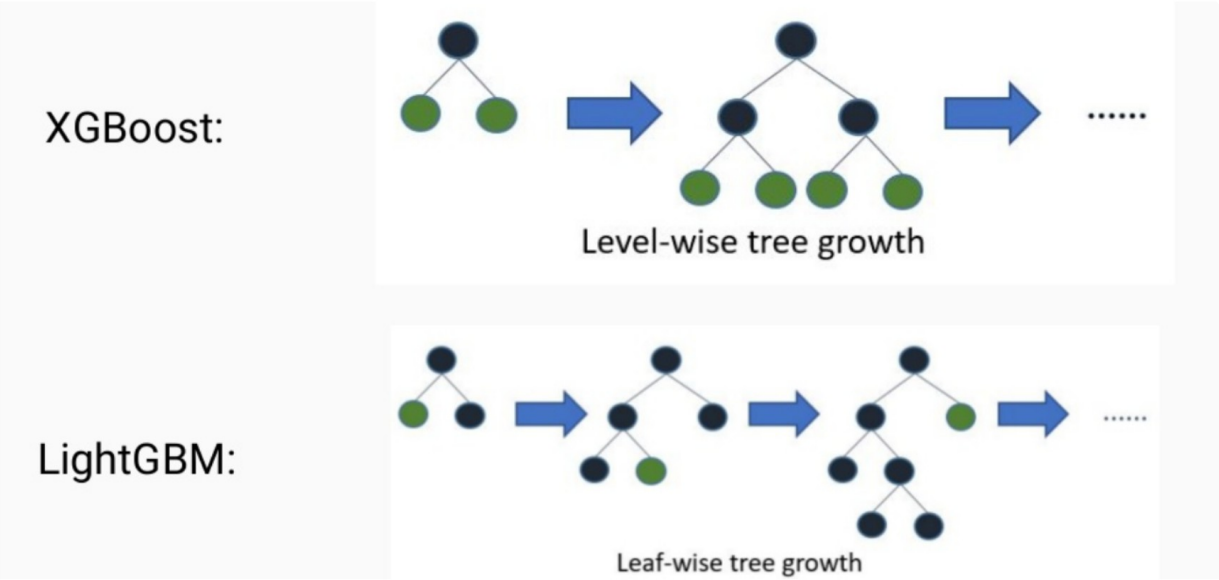
추가 설치 없이 바로

from sklearn.ensemble import HistGradientBoostingClassifier

© 2026. Lim all rights reserved.

04 LightGBM: leaf-wise 성장

XGBoost와 트리를 키우는 순서가 다르다



XGBoost 기본: level-wise

- 같은 깊이의 노드를 **한 층씩** 나눈다 (grow_policy=depthwise)
- lossguide로 바꾸면 leaf-wise도 가능

LightGBM: leaf-wise

- 손실을 **가장 많이 줄이는 리프**부터 나눈다
- 데이터가 적으면 과대적합될 수 있어 max_depth로 제한

빠른 이유

- **히스토그램** 방식이라 빠르고 메모리를 덜 쓴다

참고: https://rohitgr7.github.io/lightgbm-another-gradient-boosting/

© 2026. LDJ all rights reserved.

04 LightGBM - 매개변수 ①

트리 구조 (LightGBM 4.x 기본값, 확인일 2026-10-13)

	뜻	기본값
num_iterations	트리 개수 (scikit-learn 방식 이름: n_estimators)	**100**
learning_rate	트리마다 보정하는 세기	**0.1**
num_leaves	트리 하나의 최대 리프 수 — 복잡도의 핵심	**31**
max_depth	최대 깊이, 0 이하면 제한 없음 (그래도 leaf-wise로 자람)	**-1**
boosting	부스팅 방식: gbdt · rf · dart (goss는 4.0부터 data_sample_strategy로)	**gbdt**

© 2026. LDJ all rights reserved.

04 LightGBM - 매개변수 ②

과대적합 제어 (LightGBM 4.x 기본값, 확인일 2026-10-13)

	뜻	기본값
min_data_in_leaf	리프 하나에 필요한 최소 데이터 수 (= min_child_samples)	**20**
bagging_fraction	트리마다 쓸 데이터 비율 (= subsample) — **bagging_freq > 0** 이어야 동작 (기본 0)	**1.0**
feature_fraction	트리마다 무작위로 고를 특성 비율 (= colsample_bytree)	**1.0**
lambda_l1 · l2	L1 · L2 규제 세기 (= reg_alpha · reg_lambda)	**0.0**

© 2026. LDJ all rights reserved.

04 LightGBM - 매개변수 조정

공식 튜닝 가이드 요약

1

num_leaves부터

모델 복잡도를 정하는 가장 중요한 값. max_depth를 쓰면 num_leaves는 **2^max_depth보다 작게**

2

min_data_in_leaf로 과대적합 막기

(= min_child_samples) 너무 크면 과소적합. 데이터가 크면 수백~수천이면 충분

3

max_depth로 깊이 제한

데이터가 적을 때 leaf-wise 성장의 과대적합을 줄인다

4

그래도 과대적합이면

bagging · feature_fraction, lambda_l1 · lambda_l2, 더 많은 데이터

© 2026. LDJ all rights reserved.

05 두 모델의 비교

랜덤 포레스트 vs 그래디언트 부스팅

	랜덤 포레스트	그래디언트 부스팅
트리를 만드는 방식	서로 **독립적으로** 여러 개	앞 트리 오차를 보완하며 **차례로**
트리 개수를 늘리면	대체로 안정적 (느려질 뿐)	너무 많으면 **과대적합**될 수 있다
매개변수 조정	기본값으로도 잘 작동	learning_rate·n_estimators 조정이 중요
이럴 때	**먼저** 시도해 보는 기본 선택	예측 속도가 중요하거나 성능을 **끝까지** 끌어올릴 때

© 2026. Lim all rights reserved.

05 특성 중요도 볼 때 주의

feature_importances_ 를 그대로 믿지 말 것 (확인일 2026-10-13)

불순도 기반 (feature_importances_)

- 학습 데이터 기준이라 일반화에 쓸모없는 특성도 높게 나올 수 있다
- **값 종류가 많은** 특성(대개 연속형 숫자)을 과대평가한다

순열 중요도 (permutation_importance)

- 특성 값을 섞었을 때 점수가 **얼마나 떨어지나**로 잰다
- **테스트 데이터**로 계산할 수 있어 이 편향이 없다

코드

- from sklearn.inspection import permutation_importance
- r = permutation_importance(model, X_test, y_test, n_repeats=10, random_state=0)

© 2026. Lim all rights reserved.

06 여러 가지 모델

지금까지 배운 모델 ①

	좋은 점	주의할 점
KNN (최근접 이웃)	작은 데이터의 기본 모델, 설명하기 쉽다	데이터가 크면 느려진다
선형 모델	대용량·고차원 데이터에도 쓸 수 있다	복잡한 관계는 놓칠 수 있다
나이브 베이즈	선형 모델보다 훨씬 빠르다	분류만 가능, 정확도는 낮은 편
결정 트리	빠르고 스케일 조정 불필요, 시각화·설명이 쉽다	과대적합되기 쉽다

© 2026. Lim all rights reserved.

06 여러 가지 모델

지금까지 배운 모델 ②

	좋은 점	주의할 점
랜덤 포레스트	트리 하나보다 거의 항상 좋고 안정적, 스케일 조정 불필요	고차원 희소 데이터에 약함
그래디언트 부스팅	랜덤 포레스트보다 성능이 조금 더 좋은 경우가 많고 예측이 빠르다	학습이 느리고 매개변수 조정이 많이 필요
신경망	대용량 데이터로 매우 복잡한 모델을 만들 수 있다	매개변수·스케일에 민감, 큰 모델은 학습이 오래 걸림

© 2026. Lim all rights reserved.